Pure Autobahn BFT 완전 구현 가이드

📚 목차

1. 기본 개념 및 이론

- 01-Pure-Autobahn-BFT-개요.md - 순수 Autobahn BFT의 핵심 개념과 기존 구현과의 차이점
- 02-논문-기반-설계-원칙.md - 원본 논문의 핵심 알고리즘과 설계 원칙

2. 핵심 컴포넌트 구현

- 03-Fast-Path-순수-구현.md - Fast Path 엔진의 독립적 구현
- 04-Slow-Path-순수-구현.md - Slow Path 엔진의 독립적 구현
- 05-Autobahn-DAG-관리자.md - 자체 DAG 구조 및 순서화 알고리즘

3. 시스템 아키텍처

- 06-순수-시스템-아키텍처.md - 전체 시스템 구조와 컴포넌트 간 상호작용
- 07-적응형-모드-전환-시스템.md - 네트워크 조건에 따른 동적 모드 전환
- 08-네트워크-품질-모니터링.md - 실시간 네트워크 상태 평가 및 최적화

4. 보안 및 비잔틴 장애 허용

- 09-비잔틴-장애-허용-메커니즘.md - 비잔틴 노드 감지 및 대응
- 10-암호학적-보안-구현.md - 디지털 서명 및 메시지 인증

5. 성능 최적화

- 11-성능-최적화-전략.md - 처리량 및 지연시간 최적화
- 12-확장성-및-병렬-처리.md - 대규모 네트워크 지원 및 병렬 알고리즘

6. 구현 및 배포

- 13-개발-환경-설정.md - 개발 환경 구축 및 의존성 관리
- 14-설정-및-튜닝-가이드.md - 네트워크 환경별 최적 설정
- 15-배포-및-운영-가이드.md - 프로덕션 배포 및 모니터링

7. 테스트 및 검증

- 16-단위-테스트-전략.md - 컴포넌트별 테스트 방법론
- 17-통합-테스트-시나리오.md - 전체 시스템 검증 방법
- 18-성능-벤치마크-분석.md - 성능 측정 및 분석 도구

8. 실제 사용 사례

- 19-실제-사용-예제.md - 구체적인 구현 예제 및 코드 샘플
- 20-기존-구현과의-비교.md - Narwhal 래핑 방식과의 성능 비교

🎯 이 가이드의 목적

이 가이드는 **실제 Autobahn BFT 논문에 기반한 순수 구현**을 제공합니다:

- ✅ **완전한 독립성**: Narwhal/Bullshark 래핑 없이 순수 구현
- ✅ **논문 충실성**: 원본 논문의 알고리즘을 정확히 구현
- ✅ **실용적 최적화**: 실제 운영 환경을 고려한 엔지니어링
- ✅ **확장 가능성**: 대규모 네트워크 지원

🔄 기존 구현과의 차이점

| 측면 | 기존 래핑 방식 | 순수 구현 방식 |

|------|---------------|---------------|

| **의존성** | Narwhal/Bullshark 필요 | 완전 독립적 |

| **성능** | 래핑 오버헤드 존재 | 최적화된 직접 구현 |

| **유지보수** | 외부 의존성 관리 필요 | 자체 제어 가능 |

| **커스터마이징** | 제한적 | 완전한 제어 |

| **논문 충실성** | 부분적 구현 | 완전한 구현 |

📖 사용 방법

1. **학습 목적**: 01번부터 순서대로 읽어가며 Autobahn BFT 이해
2. **구현 목적**: 스펙 파일(.kiro/specs/pure-autobahn-bft/)과 함께 참조
3. **운영 목적**: 13-15번 문서를 중심으로 배포 및 운영 가이드 활용

각 문서는 독립적으로도 읽을 수 있지만, 전체적인 이해를 위해서는 순서대로 읽는 것을 권장합니다.